第二周知识点总结（第 5~7 讲）

> 说明：页码指对应 PDF 文本中的 `Page N`，结合第二周作业答题卡（配置归类、代码填空、训练流程实操）提炼最关键考点。

一、第 5 讲 配置系统：环境配置与训练配置怎么协作（05_.pdf）

1. 为什么需要配置系统（Page 1–2）

- 避免把大量参数（学习率、环境数量、地形、奖励权重等）硬编码在代码中。
- 核心思想：**Code defines logic, Configuration defines experiment.**
- 代码负责实现算法逻辑，配置负责决定当前实验。

2. 配置系统整体架构（Page 2–3）

一个机器人实验主要由两部分组成：

- **EnvCfg**：决定机器人所处的环境，决定机器人完成什么任务。
- **RLCfg**：决定强化学习算法如何训练，决定策略如何更新。

一句话总结：

> **EnvCfg 决定世界，RLCfg 决定学习。**

3. EnvCfg 的核心内容（Page 3–5）

- **Observation**：Policy Network 的输入，决定机器人能获取哪些状态信息（如 `base_lin_vel`、`joint_pos`）。
- **Action**：机器人作用于环境的方式（如 Joint Position、Joint Velocity、Joint Torque）。
- **Reward**：定义机器人应该学会什么（如 `tracking_reward`、`energy_penalty`）。
- **Termination**：决定 Episode 何时结束（如 `fallen`、`timeout`）。
- **Randomization**：域随机化（如质量、摩擦力、推力），提高鲁棒性，减少 Sim-to-Real Gap。
- **Curriculum**：课程学习，逐步增加训练难度（如 Flat → Slope → Rough → Obstacle）。

4. RLCfg 的核心内容（Page 5–7）

- **PPO Hyperparameters**：`learning_rate`、`gamma`、`lam`、`clip_param`、`entropy_coef` 等，控制优化过程、训练稳定性和收敛速度。
- **Runner**：控制训练流程（如 `num_steps_per_env`、`max_iterations`、`save_interval`）。
- **Logging**：控制日志记录（TensorBoard、W&B），记录 Reward、Loss、Episode Length 等。
- **Checkpoint**：控制模型保存（Policy、Optimizer、Training State），方便恢复训练。

5. 字段归类（重点参考答题卡）

- **属于 EnvCfg**：`model_file`、`sim_dt`、`render_spacing`、`max_episode_seconds`
- **属于 RLCfg**：`learning_rate`、`num_envs`、`rollouts`、`ratio_clip`

6. EnvCfg 与 RLCfg 的协作闭环（Page 8）

> EnvCfg 创建环境 → Observation → Policy 输出 Action → Environment 产生 Reward → RLCfg (PPO) 更新 New Policy。

- **EnvCfg 提供数据，RLCfg 利用数据完成策略更新。**

二、第 6 讲 注册系统：新环境如何接入框架（06_.pdf）

1. 为什么需要注册系统（Page 1）

框架需要根据 Task Name 回答：

- 对应哪个 Environment?
- 使用哪个 EnvCfg?
- 使用哪个 RLCfg?
- 加载哪个机器人模型?

Registry（注册系统）的作用：

- 建立 Task 与配置之间的映射关系。
- 统一管理所有机器人任务。
- 支持自动加载对应配置。

2. 注册系统整体流程（Page 2）

> 读取 Task Name → Registry 找到对应 Task → EnvCfg / Environment / RLCfg → 创建训练环境。

3. Task 的目录结构（Page 2–3）

每个任务通常包含：

- `__init__.py`（完成注册）
- `env_cfg.py`（环境配置）
- `rl_cfg.py`（强化学习配置）
- `xxx_env.py`（环境实现）

4. 三个核心注册项（Page 3–5）

- **注册 Environment**：告诉框架 Environment 的实现位置，创建对应环境实例（负责 `reset()`、`step()`、`reward`、`observation`）。
- **注册 EnvCfg**：`env_cfg_entry_point=HumanoidEnvCfg`。描述：**Robot lives in what world.**

- **注册 RLCfg**：`rl_cfg_entry_point=HumanoidPPORunnerCfg`。描述：**Robot learns how.**

5. 完整注册结构（Page 5）

> Task Name → Registry → Environment & EnvCfg → RLCfg → Start Training

- **缺少任何一部分，都无法启动训练。**

6. 如何创建新的 Task（Page 5–6）

1. 复制已有 Task（如 `humanoid`）。
2. 修改 Task Name。
3. 创建新的 Environment。
4. 创建新的 EnvCfg。
5. 创建新的 RLCfg。
6. 修改 `__init__.py` 完成注册。
7. 框架即可自动识别新任务，无需修改框架主体代码。

7. 代码填空与实现要点（参考答题卡）

- **装饰器注册**：`@registry.envcfg("simple_walker")`、`@registry.env("simple_walker", "np")`、`@rlcfg("simple_walker")`。
- **环境必须实现的三个抽象方法**：`apply_action()`、`update_state()`、`reset()`。
- **Import 链**：`simple_walker/__init__.py` 导入环境；分类目录的 `__init__.py` 导入任务；RL 侧 `cfgs.py` 使用 `@rlcfg` 注册配置。

三、第 7 讲 训练流程：从 train.py 到模型保存（07_.pdf）

1. 训练是如何启动的（Page 1）

命令行：`python train.py --task humanoid`

- **Train Script controls workflow, Registry loads components.**
- 训练脚本负责组织流程，注册系统负责自动加载任务。

2. 训练流程整体架构（Page 2）

> Command Line → train.py → Parse Arguments → Registry → Load Task → EnvCfg / RLCfg → Create Environment → Create Runner → Start Training。

3. 解析命令行参数（Page 2–3）

常见参数：

- `--task`：指定训练任务。
- `--num_envs`：设置并行环境数量。
- `--seed`：控制随机种子。

- `--max_iterations`：最大迭代次数。
- `--headless`：无头模式（控制运行方式）。
- 命令行参数可以**覆盖默认配置**。

4. 创建训练环境与 Runner（Page 4–5）

- **Environment 创建初始化**：Robot、Terrain、Sensors、Observation、Reward、Randomization。
- **PPORunner 自动加载**：Policy Network、Value Network、Optimizer、Replay Buffer（Rollout Buffer）。

5. 模型保存（Checkpoint）（Page 6）

- 按照 `save_interval`（如 500）自动保存。
- 保存内容：Policy、Optimizer、Runner State。
- 通常输出到 `checkpoints/` 目录。
- 用途：恢复训练、模型评估、推理部署。

6. 日志记录与输出目录（Page 7–8）

- **日志记录**：持续记录 Reward、Loss、Episode Length、Success Rate、Learning Rate。
- **输出目录**：
  - `logs/`：TensorBoard 日志、Loss 曲线、Reward 曲线。
  - `checkpoints/`：模型参数、Optimizer、Training State。
  - `videos/`：Rollout 视频、可视化结果。
  - `configs/`：EnvCfg、RLCfg、当前实验配置（便于实验复现）。

7. 训练实操验证要点（参考答题卡）

- **输出目录验证**：能看到 `runs/xxx/` 下的 `model_0.pt`、`model_50.pt` 等 checkpoint 以及 `events.out.tfevents...` 日志文件。
- **TensorBoard 分析**：启动 `uv run tensorboard --logdir runs/xxx`，观察 `Loss/entropy`、`Loss/learning_rate`、`Loss/surrogate`、`Loss/value` 曲线，并结合 `Train/mean_reward` 判断训练效果。
- **推理演示**：`uv run scripts/play.py --env xxx --rllib skrl --num-envs 1 --policy runs/.../model_299.pt`，验证环境、策略结构和 checkpoint 能否正确闭环推理。

四、第二周核心主线总结

1. **配置分离**：EnvCfg 决定世界，RLCfg 决定学习；代码定义逻辑，配置决定实验。
2. **注册映射**：Registry 建立 Task Name 到 Environment、EnvCfg、RLCfg 的映射，实现自动加载。
3. **训练闭环**：命令行 → 解析参数 → 注册系统加载任务 → 创建环境与 Runner → 训练 → 保存 Checkpoint 与日志。
4. **实操闭环**：配置修改 → 启动训练 → 查看 TensorBoard 曲线 → 加载 Checkpoint 推理验证。